

MNIST Neural Network Acceleration on GPU

Abdullah Ashfaq i22-1149
Dilawar Rasool Shah i22-0884
Junaid Asrar i22-0770

Project Report

Overview

This project involves accelerating a neural network designed for the MNIST digit recognition task using GPU acceleration via CUDA. The MNIST dataset comprises 70,000 grayscale images (28x28 pixels), which serve as a standard benchmark for evaluating neural network models. Four distinct versions (V1-V4) of the implementation were developed and compared:

- **V1:** Baseline Sequential CPU implementation
- **V2:** Naive GPU implementation
- **V3:** Optimized GPU implementation
- **V4:** Optimized GPU implementation utilizing Tensor Cores

Performance Results

The following summarizes the key performance metrics across different versions:

Version	Epoch 1 Time (s)	Epoch 2 Time (s)	Epoch 3 Time (s)	Total Training Time (s)	Test Accuracy (%)	Speedup Factor (x)
V1	7.587	7.559	7.665	22.811	96.77	1
V2	7.017	7.008	7.016	21.093	96.86	1.08
V3	4.925	4.925	4.925	14.828	96.90	1.54
V4	2.540	2.480	2.480	7.500	96.83	3

Analysis of Results

- **Baseline (V1)** performed reasonably but served as the starting point for GPU optimization.
- **Naive GPU (V2)** showed slight performance gains compared to V1, illustrating basic GPU parallelization advantages.
- **Optimized GPU (V3)** showed a considerable improvement (~32% faster) over V2 through advanced CUDA optimizations like efficient memory management, occupancy, and launch configurations.
- **Tensor Core Implementation (V4)** significantly outperformed previous versions, showing nearly double the performance gain compared to V3, achieving the lowest total training time (~7.5 seconds).

Approach for Each Step

- **V1 (Baseline Sequential Implementation):**
 - Implemented a basic fully connected neural network.
 - Ran entirely on CPU without any GPU utilization.
 - Used standard libraries (e.g., NumPy) for matrix operations.
- **V2 (Naive GPU Implementation):**
 - Directly transitioned computations to CUDA kernels.
 - Minimal changes to CPU code structure.
 - Basic parallelization with no optimization.
- **V3 (Optimized GPU Implementation):**
 - Improved kernel launch configurations to maximize parallel utilization.
 - Enhanced occupancy by tuning threads and block sizes.
 - Reduced GPU-CPU data transfer by efficient communication strategies.

- Optimized memory accesses by using shared memory and minimizing global memory access.
- **V4 (Tensor Core Optimized Implementation):**
 - Utilized NVIDIA Tensor Cores for mixed-precision matrix operations.
 - Integrated CUDA libraries specifically designed for Tensor Core usage (e.g., cuBLAS with Tensor Core support).
 - Adjusted precision to FP16/FP32 hybrid for significant performance gains.

Conclusions

The project successfully demonstrated substantial performance gains through GPU acceleration and CUDA optimization strategies. Particularly, the utilization of Tensor Cores provided remarkable efficiency, cutting down the training time by approximately 67% compared to V3 and roughly 67% to 70% compared to the naive GPU implementation (V2). This clearly illustrates the powerful impact of targeted hardware optimizations in neural network acceleration.

Github Repository

https://github.com/3astW1nd/hpc_proj/tree/main